

PROGRAMACIÓN ORIENTADA A OBJETOS

ISAAC POGGI

UNIVERSIDAD INTERNACIONAL DEL ECUADOR



INGENIERÍA EN CIBERSEGURIDAD

APRENDIZAJE AUTÓNOMO 1.

Docente:

HECTOR GUILLERMO AVALOS SILVA

Canadá, mayo 2026

ÍNDICE

1. ALCANCE DEL PROYECTO Y JUSTIFICACIÓN DISCIPLINAR.....	2
Relación con la Disciplina: Beneficios del Paradigma Funcional.....	2
2. ESTRUCTURA MODULAR DEL SISTEMA.....	2
Módulo 1: Control de Acceso (package control_acceso).....	2
Módulo 2: Catálogo de Películas (package peliculas).....	3
Módulo 3: Historial de Reproducción (package historial).....	3
3. TRANSFERENCIA METODOLÓGICA.....	4

SISTEMA DE STREAMING

1. ALCANCE DEL PROYECTO Y JUSTIFICACIÓN DISCIPLINAR

El objetivo de este proyecto es diseñar y estructurar el backend lógico para una plataforma de gestión de streaming de video. El sistema se fundamenta en los principios de la programación funcional utilizando el lenguaje Go.

Relación con la Disciplina: Beneficios del Paradigma Funcional

La elección de la programación funcional para este dominio de estudio responde a dos necesidades críticas de las plataformas modernas de streaming:

1. **Concurrencia Segura:** Al utilizar datos inmutables y funciones puras, eliminamos los estados compartidos. Esto permite que millones de usuarios consulten el catálogo simultáneamente sin riesgo de corrupción de datos.
2. **Mantenibilidad:** Cada función se comporta de manera predecible, facilitando las pruebas unitarias y el aislamiento de errores.

2. ESTRUCTURA MODULAR DEL SISTEMA

Módulo 1: Control de Acceso (package control_acceso)

Encargado de la seguridad, autenticación y registro de cuentas.

- **Estructuras de Datos:**

Go

```
type Usuario struct {  
    Email string  
    Contraseña string  
}
```

- **Funcionalidades y Lógica Pura:**

- **ValidarAcceso(usuarios []Usuario, email, pass string) int:** Compara credenciales y retorna estados determinados (0: Éxito, 1: No existe, 2: Contraseña errónea) sin modificar la lista.
- **RegistrarUsuario(usuarios []Usuario, nuevo Usuario) ([]Usuario, bool):** Aplica la inmutabilidad. Si el usuario no existe, genera y retorna un nuevo slice combinando los datos previos con el nuevo registro mediante la función append, manteniendo intacto el contenedor original.
- **SolicitarRecuperacion(usuarios []Usuario, email string) bool:** Valida la existencia del identificador antes de disparar eventos simulados de restauración.

Módulo 2: Catálogo de Películas (package peliculas)

Administra los metadatos de los contenidos audiovisuales disponibles.

- **Estructuras de Datos:**

Go

```
type Pelicula struct {  
    Titulo    string  
    Genero    string  
    FechaSubida time.Time // Paquete nativo para gestión cronológica  
}
```

- **Funcionalidades y Lógica Pura:**

- **OrdenarPorNovedad(peliculas []Pelicula) []Pelicula:** Diseñada para la experiencia de usuarios nuevos. Para evitar efectos secundarios en el catálogo global, la función realiza una copia local de la estructura (copy) y la ordena cronológicamente utilizando sort.Slice junto con el método de comparación estricta .After().

Módulo 3: Historial de Reproducción (package historial)

Sigue la actividad del usuario para personalizar la interfaz de consumo.

- **Estructuras de Datos:**

Go

```
type HistorialVideo struct {  
    Email    string  
    IDPelicula string  
    TiempoActual int // Progreso de reproducción en segundos  
    TiempoTotal int // Duración total del archivo de video  
}
```

- **Funcionalidades y Lógica Pura:**

- **EsUsuarioNuevo(historial []HistorialVideo, email string) bool:** Determina la ausencia de registros de consumo para redirigir el flujo hacia las novedades del catálogo.
- **SepararHistorial(historial []HistorialVideo, email string) ([]HistorialVideo, []HistorialVideo):** Ejecuta un filtro matemático de transferencia. Utiliza la evaluación del progreso ($\text{TiempoActual} == \text{TiempoTotal}$ para Volver a ver, y $\text{TiempoActual} < \text{TiempoTotal}$ para Seguir viendo) devolviendo colecciones de datos completamente nuevas y aisladas.

3. TRANSFERENCIA METODOLÓGICA

La arquitectura lógica y el principio de inmutabilidad diseñados en este proyecto no son solo teóricos, sino que representan fielmente el funcionamiento de plataformas globales de streaming como Netflix.

En una aplicación real, el sistema transfiere y procesa los datos siguiendo exactamente esta misma metodología funcional:

- **Gestión de Nuevos Usuarios:** Al igual que nuestra función OrdenarPorNovedad, cuando un usuario crea una cuenta en Netflix, la plataforma detecta la ausencia de un historial activo y recurre a un ordenamiento cronológico puro para mostrar los estrenos más recientes de su catálogo regional.
- **Segmentación del Flujo de Consumo:** La división matemática que propusimos en SepararHistorial es el núcleo que define la interfaz del usuario. Si un usuario reproduce un video y lo pausa a la mitad, la condición matemática ($\text{TiempoActual} < \text{TiempoTotal}$) transfiere automáticamente ese contenido a la fila de "Continuar viendo".
- **Consumo Completado:** En el momento en que el progreso iguala a la duración total del archivo, el sistema aplica la resta lógica para remover el video de la lista de pendientes y moverlo al módulo de "Ver de nuevo" o sugerir contenido similar. Todo esto sucede a través de funciones puras en el backend que transforman los datos en tiempo real sin alterar el catálogo base de la empresa.

Video explicativo del sistema elegido.

[Grabando-20260522_230232.webm](#)

Repositorio en GitHub:

[IsaacPoggi89/streaming-funcional-go: Etapa 1: Planeación y lógica funcional para sistema de streaming](#)